sercon

User Manual

Version 0.4.10

2026-05-26

Repository · https://github.com/codedeviate/sercon

sercon manual

Long-form reference for the `pkg/scriptengine` library, the `sercon` CLI, the built-in script API, and the JavaScript runtime surface that `scriptengine` exposes via goja and goja_nodejs. Examples are runnable — save snippets as `.ts` files and execute with `sercon file.ts` unless otherwise noted.

Table of contents

---

1. Overview

`sercon` is a Go program that runs TypeScript files. It's built from two parts:

- `pkg/scriptengine` — a library you embed in your own Go code. You register Go-callable bindings, then call `Run` / `RunFile` to execute a `.ts` script that talks to them.
- `cmd/sercon` — a thin CLI built on the library, useful for ad-hoc test scripts and as a working example of every binding kind.

The runtime is pure Go: goja is the JS engine, esbuild (used as a Go library) is the TS→JS transpiler, and goja_nodejs provides Promises, `setTimeout`, `console`, and CommonJS `require`. No cgo, no Node.

2. Quickstart

Install:

```
go install github.com/codedeviate/sercon/cmd/sercon@latest
```

Run one of the bundled examples:

```
sercon examples/scripts/smoke.ts examples/scripts/async.ts
```

…or run them all via `make demo`. `examples/scripts/` ships a runnable `.ts` (or `.tsx`) file per feature area — `hash.ts`, `strings.ts`, `path-and-time.ts`, `default-export.ts`, `tsx-demo.ts`, and the original `smoke.ts` / `async.ts` / `hang.ts` (`hang.ts` is the timeout demo and intentionally exits non-zero).

Generate a declaration file for editor autocomplete:

```
sercon --emit-dts api.d.ts
```

Embed in your own Go program:

```
eng := scriptengine.New(scriptengine.Options{
    Timeout:    5 * time.Second,
    ScriptRoot: "./scripts",
})
eng.Register("upper", strings.ToUpper)
val, err := eng.RunFile(ctx, "scripts/main.ts")
```

3. Library API — `pkg/scriptengine`

`Engine`, `Options`, `New`

```
type Options struct {
    Timeout        time.Duration // wall-clock limit per Run; 0 disables
    ScriptRoot     string        // base for require/import resolution
    DisableConsole bool          // turn off the goja_nodejs console module
}

func New(opts Options) *Engine
```

`Engine` is the host of all registered bindings. Construct it once, then call `Run` / `RunFile` as many times as needed — each call gets a *fresh* `*goja.Runtime` so script state never leaks between invocations.

`ScriptRoot` defaults to the working directory at `New` time if left empty. Both `Timeout` and `ScriptRoot` can be overridden on a per-Run basis only by constructing a fresh `Engine`; see Out-of-scope for the per-Run override on the roadmap.

Registering bindings

Function	Use when…
`Register(name, value)`	The binding is a pure function, struct, or primitive that doesn't need access to the runtime.
`RegisterNamespace(name, members map[string]any)`	You want to expose `name.x`, `name.y` without defining a Go struct.
`RegisterConstructor(name, ctor)`	The binding is a Go function whose return value should be treated as a class in the emitted `.d.ts`. (Runtime semantics today are identical to `Register`; the d.ts emitter is the only differentiator.)
`RegisterFactory(name, factory func(vm, loop) any)`	The binding needs the per-Run `*goja.Runtime` or `*eventloop.EventLoop` in scope (typical for Promise-returning I/O).
`RegisterNamespaceFactory(name, factory)`	Same, but the factory returns the full member map.

Example — synchronous function binding:

```
eng.Register("greet", func(name string) string { return "hi " + name })
```

Example — namespace with a sync member:

```
eng.RegisterNamespace("math2", map[string]any{
    "square": func(n float64) float64 { return n * n },
    "pi":     3.14159,
})
```

Example — async (Promise-returning) binding via `PromisifyAsync`:

```
eng.RegisterFactory("httpGet", func(vm *goja.Runtime, loop
*eventloop.EventLoop) any {
    return scriptengine.PromisifyAsync(vm, loop,
        func(ctx context.Context, call goja.FunctionCall) (map[string]any,
error) {
            url := call.Argument(0).String()
            resp, err := http.Get(url)
            if err != nil {
                return nil, err
            }
            defer resp.Body.Close()
            body, _ := io.ReadAll(resp.Body)
            return map[string]any{"status": resp.StatusCode, "body":
string(body)}, nil
        })
})
```

`Run`, `RunFile`

```
func (e *Engine) Run(ctx context.Context, name, source string, opts
...RunOption) (goja.Value, error)
func (e *Engine) RunFile(ctx context.Context, path string, opts
...RunOption) (goja.Value, error)
```

`Run` executes `source` as an entry-script TS. `name` is used in stack traces. The returned value is currently always `undefined` — top-level expression capture is on the backlog.

Both methods respect `Options.Timeout` *and* `ctx` cancellation, whichever fires first; the resulting error is either `ErrScriptTimeout`, `ctx.Err()`, or the underlying JS exception.

Per-Run options

```
type RunOption func(*runConfig)

func WithScriptRoot(dir string) RunOption
```

Reuse a single Engine across many scripts that each live in their own directory by passing `WithScriptRoot(dir)` to `Run` / `RunFile`. The override applies only to this call; `Options.ScriptRoot` is used for every other Run.

```
_, _ = eng.Run(ctx, "main.ts", source,
scriptengine.WithScriptRoot("/path/to/run42"))
```

`Reset`

```
func (e *Engine) Reset()
```

Clears every registered binding. Useful when a long-lived Engine is reused across unrelated batches of scripts that each want a clean global namespace. **Not safe to call concurrently with Run / RunFile.**

`WriteTypes`

```
func (e *Engine) WriteTypes(w io.Writer) error
```

Emits a `.d.ts` describing the registered surface. See section 13. Type generation for what the mapping looks like.

`PromisifyAsync[T]` and `AsyncBinding`

```
type AsyncBinding struct {
    Func         func(goja.FunctionCall) goja.Value
    TSReturnType string
}

func PromisifyAsync[T any](vm *goja.Runtime, loop *eventloop.EventLoop,
    work func(ctx context.Context, call goja.FunctionCall) (T, error),
) AsyncBinding
```

`PromisifyAsync` turns blocking Go work into a JS Promise. It launches the work in a goroutine, parks a `SetTimeout` sentinel so `loop.Run` doesn't return early, and schedules the resolve/reject back onto the event loop. **Required** for any Promise-returning binding — `RunOnLoop` alone is not counted as a live job by the event loop.

`PromisifyAsync` returns an `AsyncBinding` carrier rather than the bare callback. The engine unwraps it to `.Func` at `vm.Set` time so goja's special-case detection of `func(goja.FunctionCall) goja.Value` host-callbacks still fires; the `.d.ts` emitter reads `.TSReturnType` to emit `Promise<T>` (mapped from the generic `T` via reflect at construction time) instead of the previous `unknown`. Hosts assigning the result into a `map[string]any` for `RegisterNamespace` / `RegisterNamespaceFactory` get the unwrap recursively too — no manual work required.

Version

```
import "github.com/codedeviate/sercon/pkg/scriptengine"

fmt.Println(scriptengine.Version) // "0.1.0"
```

Bumped in lockstep with the git tag.

4. CLI — `sercon`

```
sercon [flags] <script.ts> [script.ts ...]
sercon [flags] -               # read entry script from stdin
sercon --examples | --help | --version
```

Flag	Purpose
`-timeout DURATION`	Wall-clock limit per script (default `10s`; `0` disables).
`-root DIR`	Override the script root for `require`/`import` resolution.
`-emit-dts PATH`	Write the example bindings' `.d.ts` to `PATH` and exit.
`-v`	Verbose: trace the rewritten entry-script JS and each module resolution to stderr; also print duration on failure.
`-h`, `--help`	In-depth colorized help.
`--examples`	In-depth colorized walkthrough of every feature.
`--version`	Print the engine version (plus goja/esbuild build-info versions).

Each positional argument is either a path to a `.ts` / `.tsx` file or `-` to read an entry script from standard input:

```
echo 'api.log(1 + 2);' | sercon -
sercon prelude.ts -                # prelude then stdin
```

Exit codes are distinct per failure type so shells can react sensibly. When several scripts run, the highest applicable code wins:

Code	Meaning
`0`	every script passed.
`1`	CLI usage error (unknown flag, missing script argument, …).
`2`	at least one script failed to transpile — never ran.
`3`	at least one script timed out (`-timeout`) or was context-cancelled.
`4`	at least one script ran and threw a JS exception.

`-v` writes lines prefixed with `[sercon] ` to stderr. The traces include the full rewritten entry-script JS (the form goja actually runs, after the ESM→CJS rewrite + async IIFE wrapper) and every module-resolution event, so debugging an unexpected resolve target or a mis-rewritten import is straightforward.

The CLI registers a single `api` namespace; see the next section.

5. Built-in script `api`

The CLI exposes the following globals to every script:

```
declare const api: {
  log(...args: unknown[]): void;

  assert: {
    equal(actual: unknown, expected: unknown, msg?: string): void;
    ok(cond: unknown, msg?: string): void;
  };

  http: {
    get(url: string): Promise<{ status: number; body: string }>;
    post(url: string, body?: string): Promise<{ status: number; body:
string }>;
  };

  time: {
    nowMs(): number;
    sleep(ms: number): Promise<void>;
    format(unixMs: number, fmt: string, tz?: string): string;
  };

  env: {
    get(name: string): string | undefined;
  };

  str: {
    trim(s: string, mask?: string): string;
    ltrim(s: string, mask?: string): string;
    rtrim(s: string, mask?: string): string;
    reverse(s: string): string;
    stripHtml(s: string): string;
    nl2br(s: string, xhtml?: boolean): string;
    br2nl(s: string): string;
    base64Encode(s: string): string;
    base64Decode(s: string): string;
    urlEncode(s: string): string;
    urlDecode(s: string): string;
    htmlEntityDecode(s: string): string;
    pad(s: string, len: number, padChar?: string, side?: "left" | "right" |
"both"): string;
    lpad(s: string, len: number, padChar?: string): string;
    rpad(s: string, len: number, padChar?: string): string;
    sprintf(fmt: string, ...args: unknown[]): string;
    printf(fmt: string, ...args: unknown[]): void;
    normalizeNewlines(s: string, style: "lf" | "crlf" | "cr"): string;
  };

  path: {
    dirname(p: string): string;
```

```
    basename(p: string, suffix?: string): string;
  };

  hash: {
    md5(data: string): string;
    sha1(data: string): string;
    sha256(data: string): string;
    sha384(data: string): string;
    sha512(data: string): string;
    sha3_256(data: string): string;
    sha3_512(data: string): string;
    blake3(data: string): string;
    crc32(data: string): string;
  };

  compression: {
    algos(): string[];
    compress(
      algo: "gzip" | "deflate" | "zlib" | "bzip2" | "zstd" | "brotli" |
"lz4" | "xz" | "snappy",
      data: string | ArrayBuffer | Uint8Array,
    ): Promise<ArrayBuffer>;
    decompress(
      algo: "gzip" | "deflate" | "zlib" | "bzip2" | "zstd" | "brotli" |
"lz4" | "xz" | "snappy",
      data: string | ArrayBuffer | Uint8Array,
    ): Promise<ArrayBuffer>;
  };

  net: {
    tcp(target: string, opts?: { timeout?: number; port?: string }):
Promise<{
      host: string; port: number; ip: string; latencyMs: number;
    }>;
    dns(host: string, opts?: { types?: ("a" | "aaaa" | "mx" | "txt" |
"cname" | "ns")[] }): Promise<{
      a?: string[]; aaaa?: string[];
      mx?: { preference: number; host: string }[];
      txt?: string[]; cname?: string; ns?: string[];
    }>;
    tls(target: string, opts?: { timeout?: number }): Promise<{
      cn: string; issuer: string;
      notBefore: string; notAfter: string;
      daysRemaining: number;
      dnsNames: string[];
      serialNumber: string;
      fingerprintSha256: string;
    }>;
    ntp(host: string, opts?: { timeout?: number; port?: number }):
```

```
Promise<{
      serverTime: string;
      offsetMs: number;
      rttMs: number;
      stratum: number;
      referenceTime: string;
      rootDelayMs: number;
      rootDispersionMs: number;
    }>;
    whois(domain: string, opts?: { timeout?: number }): Promise<{
      raw: string;
      domain?: {
        name: string; punycode?: string;
        whoisServer?: string;
        nameServers?: string[]; status?: string[];
        dnssec?: boolean;
        createdDate?: string; updatedDate?: string; expirationDate?:
string;
      };
      registrar?: { name?: string };
    }>;
  };

  email: {
    spf(domain: string): Promise<
      | { present: false }
      | {
          present: true;
          record: string;
          mechanisms: string[];
          allPolicy: "pass" | "fail" | "softfail" | "neutral" | "";
        }
    >;
    dmarc(domain: string): Promise<
      | { present: false }
      | {
          present: true;
          record: string;
          tags: Record<string, string>;
          policy?: string;
          subdomain?: string;
          percent?: string;
          rua?: string;
          ruf?: string;
        }
    >;
    mtaSts(domain: string): Promise<
      | { present: false }
      | {
```

```
          present: true;
          record: string;
          txt: { v: string; id: string };
          policy?: {
            version?: string;
            mode?: "enforce" | "testing" | "none" | string;
            mx?: string[];
            maxAge?: number | string;
          };
          policyError?: string;
        }
    >;
    tlsRpt(domain: string): Promise<
      | { present: false }
      | {
          present: true;
          record: string;
          tags: Record<string, string>;
          rua?: string;
        }
    >;
    bimi(domain: string, opts?: { selector?: string }): Promise<
      | { present: false; selector: string }
      | {
          present: true;
          selector: string;
          record: string;
          tags: Record<string, string>;
          l?: string;
          a?: string;
        }
    >;
    all(domain: string): Promise<{
      domain: string;
      spf:    unknown;
      dmarc:  unknown;
      mtaSts: unknown;
      tlsRpt: unknown;
      bimi:   unknown;
    }>;
  };
};
```

Examples:

```
api.log("hello", 1 + 2);
api.assert.equal(api.time.nowMs() > 0, true);

const r = await api.http.get("https://example.com");
api.assert.ok(r.status === 200, `expected 200, got ${r.status}`);

await api.time.sleep(50);
const home = api.env.get("HOME") ?? "(none)";

api.log(api.hash.sha256("abc"));
// → ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
```

HTTP bindings use `net/http` with a 5-second default per-request timeout and surface real `Promise<…>` values through the event loop. They are *not* mockable from JS — they go to the real network.

Compression bindings (`api.compression.*`) cover nine pure-Go algorithms behind a uniform interface. Inputs are either strings (interpreted as UTF-8 byte sequences) or any `ArrayBuffer` / `Uint8Array` ; outputs are `ArrayBuffer` (wrap with `new Uint8Array(...)` to iterate). Every algorithm round-trips:

Algorithm	Library
`gzip` / `deflate` / `zlib`	stdlib `compress/*`
`bzip2`	stdlib `compress/bzip2` for read; `github.com/dsnet/compress/bzip2` for write
`zstd`	`github.com/klauspost/compress/zstd`
`brotli`	`github.com/andybalholm/brotli`
`lz4`	`github.com/pierrec/lz4/v4`
`xz`	`github.com/ulikunitz/xz`
`snappy`	`github.com/golang/snappy`

`api.compression.algos()` returns the supported names so scripts can iterate without hard-coding the list. Unknown algorithm names throw a clear error rather than silently returning empty bytes.

Hash bindings interpret the input as a UTF-8 byte sequence and return lowercase hex. SHA-3 functions are `sha3_256` / `sha3_512` (the IETF spec uses the underscore so the JS name matches recon's). BLAKE3 uses the upstream `lukechampine.com/blake3` reference implementation with a 32-byte output. `crc32` is the IEEE polynomial, zero-padded to 8 hex chars.

String utilities (`api.str.*`) follow PHP-/recon-style semantics where they differ from JS:

- `trim` / `ltrim` / `rtrim` accept an optional mask string; **any character in the mask** is stripped. Default mask is whitespace (`" \t\n\r\v\f"`).
- `reverse` is rune-aware, so `reverse("café")` is `"éfac"`. Grapheme clusters are not handled — that's a separate problem.
- `nl2br` emits `<br>` by default; pass `true` as the second arg for XHTML-style `<br/>`.
- `urlEncode` / `urlDecode` use the form encoding (`+` for space). For path-segment encoding, use `encodeURIComponent` instead — that one is provided by goja natively.
- `pad` / `lpad` / `rpad` follow recon's `str_pad`: `pad` defaults to side `"right"`; `"left"` and `"both"` are also accepted.
- `sprintf` / `printf` are thin wrappers over Go's `fmt`, so the verb set is Go's (`%s`, `%d`, `%x`, `%.2f`, `%v`, `%t`, `%q`, etc.) — not PHP's. `printf` writes to stdout.
- `normalizeNewlines` canonicalises any mix of `\r\n`, `\r`, and `\n` to the requested style.

Path utilities (`api.path.*`) are POSIX (forward-slash). On Windows either pass forward-slash paths or convert separators yourself.

Time formatting (`api.time.format(unixMs, fmt, tz?)`) takes Unix milliseconds (symmetric with `api.time.nowMs()`) and a small strftime token set: `%Y %y %m %d %H %M %S %F %T %j %A %a %B %b %z %Z %%`. Day and month names are in English. Pass a third-argument IANA zone name (e.g. `"UTC"`, `"America/New_York"`) to render in that zone; if omitted, the host's `time.Local` is used. Unknown `%X` tokens are emitted verbatim so a typo is visible rather than silently dropped.

Protocol probes (`api.net.*`) are stdlib-backed and hit the real network. All three return Promises (via `PromisifyAsync`) and accept an optional `{ timeout: <ms> }` second arg; the default is 5 seconds.

- **`tcp(target, opts?)`** — dials TCP, measures round-trip from resolution to handshake. `target` is `"host:port"` or just `"host"` (with `opts.port` overriding the default `80`). Resolves to the remote IP, port, and `latencyMs`.
- **`dns(host, opts?)`** — looks up A, AAAA, MX, TXT, CNAME, and NS records via `net.Resolver`. Pass `opts.types` to scope the query; empty record sets are omitted from the result object so scripts can probe membership with `if ("mx" in result)`.
- **`tls(target, opts?)`** — connects with `InsecureSkipVerify` so even expired or hostname-mismatched certs come back inspectable. Returns the leaf cert's CN, issuer CN, validity window, `daysRemaining`, `dnsNames`, serial number, and a SHA-256 fingerprint (lowercase hex). Default port is `443`. Hosts that care about validity should re-validate the cert themselves.
- **`ntp(host, opts?)`** — queries an NTPv4 server (UDP 123 by default) via [`github.com/beevik/ntp`]. Returns the server time (ISO 8601), local-clock offset, round-trip time, stratum, reference time, and the root delay / dispersion values from the server's response. All durations are in milliseconds with sub-millisecond precision.
- **`whois(domain, opts?)`** — two-hop WHOIS lookup (IANA → registrar's whois server) via [`github.com/likexian/whois`] with parsing through [`github.com/likexian/whois-parser`]. `raw` is always populated; `domain` and `registrar` are best-effort parsed

views (some TLDs the parser doesn't understand will return only `raw`). The whois library doesn't accept a `context.Context` — `opts.timeout` is plumbed through its own per-client setting and the engine's `Options.Timeout` watchdog catches the outer call.

Email-authentication probes (`api.email.*`) read DNS records and surface the published policy. They all return `{ present: false }` when the relevant record is absent (NXDOMAIN or no TXT record matching the marker prefix), so scripts can write a single presence-check pattern across the family.

- **`spf(domain)`** — queries `TXT(<domain>)` and returns the first record starting with `v=spf1`. `mechanisms` is the tokenised sequence after the version prefix; `allPolicy` summarises the trailing `all` qualifier (`"pass"` for `all` / `+all`, `"fail"` for `-all`, `"softfail"` for `~all`, `"neutral"` for `?all`, empty string when no `all` is present).

- **`dmarc(domain)`** — queries `TXT(_dmarc.<domain>)` and parses the `v=DMARC1; key=val; …` form into a flat tag map (keys are case-folded; values retain internal whitespace and commas). `policy` / `subdomain` / `percent` / `rua` / `ruf` are surfaced separately because those are the fields most scripts care about.

- **`mtaSts(domain)`** — looks up `TXT(_mta-sts.<domain>)` and, on success, fetches the policy file at `https://mta-sts.<domain>/.well-known/mta-sts.txt`. The TXT carries a versioned `id` for change detection; the policy file is where the actual `mode` + `mx` list live. RFC 8461 format (line-based, `key: value`, `mx:` repeats) is parsed inline. Policy-fetch failures (TLS errors, 4xx, timeout) don't fail the binding — the TXT part is still returned, and the fetch error surfaces under `policyError` so scripts can decide what to do.

- **`tlsRpt(domain)`** — looks up `TXT(_smtp._tls.<domain>)` and parses the `v=TLSRPTv1; rua=…` form into a tag map. `rua` is surfaced separately because that's the actionable bit.

- **`bimi(domain, opts?)`** — looks up `<selector>._bimi.<domain>`, selector defaulting to `default`. Surfaces the logo URL (`l`) and VMC URL (`a`) from the tag map.

- **`all(domain)`** — runs every probe above in parallel via goroutines and returns a single aggregate object keyed by probe name. Per-probe failures don't fail the aggregate — they surface under `<probe>.error` so a partial result is still useful (e.g. SPF + DMARC found, MTA-STS policy fetch timed out).

6. JavaScript runtime built-ins (goja)

`scriptengine` runs on goja, which implements **ES5.1 + a large subset of ES6+**. The following are present and behave per spec:

Globals

- `Object`, `Array`, `String`, `Number`, `Boolean`, `BigInt`
- `Math`, `Date`, `JSON`, `RegExp`
- `Error`, `TypeError`, `RangeError`, `SyntaxError`, `ReferenceError`, `EvalError`, `URIError`
- `Promise` (provided via goja's native implementation; resolution microtasks are pumped by the event loop)
- `Symbol` (partial: well-known symbols `Symbol.iterator`, `Symbol.asyncIterator`, etc., are supported)
- `Proxy`, `Reflect` (partial)

```
api.log(Math.PI.toFixed(4), Math.max(1, 9, 3));
api.log(new Date().toISOString());
api.log(JSON.stringify({ a: 1, b: [2, 3] }));
api.log("abc".repeat(3), "abc".padStart(6, "_"));
```

Collections

- `Map`, `Set`, `WeakMap`, `WeakSet`

```
const counts = new Map<string, number>();
for (const ch of "banana") counts.set(ch, (counts.get(ch) ?? 0) + 1);
api.log([...counts]); // [["b",1],["a",3],["n",2]]
```

Typed arrays

- `ArrayBuffer`, `DataView`
- `Int8Array`, `Uint8Array`, `Uint8ClampedArray`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `Float32Array`, `Float64Array`

```
const buf = new ArrayBuffer(4);
new DataView(buf).setUint32(0, 0xCAFEBABE);
api.log(new Uint8Array(buf)[0].toString(16)); // ca
```

Iteration and generators

- `for…of`, `for…in`, spread, destructuring
- Generator functions (`function*`)
- Async generators (`async function*`) — supported via goja's event loop

```
function* fib() {
  let [a, b] = [0, 1];
  while (true) {
    yield a;
    [a, b] = [b, a + b];
  }
}
const it = fib();
const first10 = Array.from({ length: 10 }, () => it.next().value);
api.log(first10);
```

Not (yet) provided

- `fetch` — use `api.http.*` from the CLI, or register your own binding.
- `Worker`, threading primitives.
- DOM globals (`window`, `document`).
- Native ES modules (`import`/`export` are *transpiled* to CommonJS at load time — see section 8).

7. Async runtime additions (goja_nodejs)

The event loop bundles a small Node-compatible runtime on top of goja:

Timers

```
setTimeout(() => api.log("after 50ms"), 50);
setInterval(() => api.log("tick"), 100);
setImmediate(() => api.log("next tick"));
clearTimeout(t); clearInterval(i); clearImmediate(im);
```

The event loop holds the script alive while *any* timer is pending. If your script ends with only `setTimeout` callbacks scheduled, the run will wait for them to fire before returning.

`console`

```
console.log("info");
console.error("oops");
console.warn("careful");
console.info("fyi");
console.debug("trace");
```

Disable with `Options.DisableConsole = true` if you want a clean global namespace.

`require`

CommonJS require, with TS-aware extension fallback added by sercon (see section 10):

```
const helpers = require("./helpers");
helpers.doThing();
```

Both `require("./foo")` and `import { x } from "./foo"` resolve to the same module instance per Run; esbuild rewrites `import` to `require` at transpile time.

8. TypeScript support

TS is transpiled by esbuild in-process. There is no `tsc`, no `tsconfig.json`, no type-checking — types are erased and the resulting JS is what executes. Practical implications:

- All TS syntax esbuild supports is allowed: type annotations, `interface`, `type`, generics, enums (compiled to objects), namespaces, decorators (legacy & TC39 stage 3).
- `import type { X } from "y"` is stripped entirely (no runtime require).
- TS errors are *not* surfaced as build errors — only esbuild parse errors are. Run a separate `tsc --noEmit` in CI if you want type enforcement.
- `tsconfig.json` is **not** consulted.

`.tsx` and `.jsx` are supported via esbuild's `LoaderTSX` / `LoaderJSX`, including for required modules resolved by the engine's extension fallback. JSX has no React runtime in scope by default — either set the factory at the file level with an `@jsx` pragma:

```
/** @jsx h */
function h(tag: string, props: any, ...children: any[]) {
  return { tag, props: props ?? {}, children };
}
export const Box = (label: string) => <div className="box">{label}</div>;
```

…or provide a `React.createElement`-compatible binding from Go and rely on esbuild's default pragma. ESM `export default <value>` also works through the entry-script rewriter's `__esModule ? .default : m` unwrap, so `import answer from "./mod"` resolves to the default export.

9. Top-level `await`

Top-level `await` works in entry scripts:

```
const r = await api.http.get("https://example.com");
api.log(r.status);
```

How it works under the hood: esbuild rejects top-level await with the `cjs` output format, so the engine transpiles the *entry* script with `format=esm`, then rewrites `import` statements to `require()` calls and wraps the rest of the body in:

```
;(async () => { /* your transpiled body */ })().then(__resolve, __reject);
```

`__resolve` and `__reject` are bindings the engine sets on the runtime to capture the final settlement. *Required-module* code (anything loaded through `require`) is transpiled with `format=cjs` and does **not** support top-level await — wrap in `(async () => …)()` yourself if you need it inside a module.

10. Module resolution

The engine plugs a custom source loader into `goja_nodejs/require` that adds:

1. Direct path: if the requested file exists on disk, use it. `.ts` / `.tsx` files are transpiled on the fly.
2. **Extension fallback** when the request has no extension: try `.ts`, `.tsx`, `.js`, `.cjs`, `.mjs`, `.json` in that order.
3. **`.js` → `.ts` swap**: if a path ending in `.js` / `.cjs` / `.mjs` is requested but the literal file doesn't exist, the same path ending in `.ts` (or `.tsx`) is tried. Handles `package.json` `main` fields that point at compiled output where only the TypeScript source is on disk.
4. **`package.json` `source` preference**: when reading a `package.json`, if a `source` field is present and points at an existing `.ts`/`.tsx` file, `main` is rewritten to that path before the registry consumes it. Matches the convention used by parcel, microbundle, and similar bundlers to mark the TS source of truth.
5. **Directory index**: if the resolved path is a directory, try `index.ts`, `index.tsx`, `index.js`.

The base directory follows Node's CommonJS rules — relative imports resolve against the requiring module's directory, courtesy of goja_nodejs.

```
// All resolve to ./helpers/assert.ts:
import { check } from "./helpers/assert";
import { check } from "./helpers/assert.ts";
const { check } = require("./helpers/assert");
const { check } = require("./helpers/assert.js");
```

JSON imports work out of the box via either the ESM-style default import (esbuild rewrites it to a require) or a direct `require`:

```
import data from "./data.json";
const r = require("./data.json");
```

`node_modules` lookup *works* via the upstream registry, but the source loader doesn't currently transpile through that path — keep TS helpers under `ScriptRoot`.

11. Timeouts and cancellation

Two mechanisms interrupt a running script:

- `Options.Timeout` — wall-clock limit per `Run`. Exceeding it returns `scriptengine.ErrScriptTimeout`.
- `ctx` passed to `Run` / `RunFile` — cancelling the context returns `ctx.Err()` (typically `context.Canceled` or `context.DeadlineExceeded`).

Both call `vm.Interrupt(...)` and `loop.Terminate()` from a watcher goroutine. This works for **sync** JS — including `while(true){}` — because the goja VM checks the interrupt flag between bytecode steps. A host Go binding that's blocked in cgo or a long syscall isn't interruptible mid-call; if you write such a binding, plumb the engine `ctx` through to it yourself.

```
eng := scriptengine.New(scriptengine.Options{Timeout: 200 *
time.Millisecond})
_, err := eng.Run(ctx, "loop.ts", `while (true) {}`)
// err is scriptengine.ErrScriptTimeout, returned ~200-300ms after Run.
```

12. Error semantics

- A Go binding returning `(T, error)` automatically throws as a JS exception when `err != nil`. The exception's `.message` is `err.Error()`.

  ```
  try { mayFail(); } catch (e) { api.log(String(e)); }
  ```

- A Go binding that `panic`s with a `*goja.Object` (e.g. `vm.NewGoError(...)`) does the same.

- A script throwing a JS error out of the top level surfaces from `Run` as a Go `error` whose message is the JS error's `.message`.

- Timeout errors satisfy `errors.Is(err, scriptengine.ErrScriptTimeout)`. Cancellation errors satisfy `errors.Is(err, context.Canceled)` or `context.DeadlineExceeded`.

13. Type generation (.d.ts)

`Engine.WriteTypes(w)` walks the registered bindings and emits a TypeScript declaration file. The mapping table (abbreviated):

Go type	TS type
`string`	`string`
any numeric / `float64`	`number`
`bool`	`boolean`
`[]T`	`T[]`; `[]byte` → `Uint8Array`
`map[string]T`	`Record<string, T>`
`*T`	`T` (transparent unwrap)
`error` (single return)	omitted (collapses to `void`)
`(T, error)` (tuple return)	`T`
`interface{}` / `any`	`unknown`
`goja.FunctionCall` parameter	folded into `(...args: unknown[])`
`goja.Value` return	`unknown`
`scriptengine.AsyncBinding` value (from `PromisifyAsync[T]`)	`Promise<T>`
self-referential types	`unknown` (cycle detection bails after depth 4)

```
sercon --emit-dts api.d.ts
```

In your editor, place the resulting `.d.ts` next to your scripts (or reference it via `/// <reference path="..." />`) for autocomplete.

14. Limitations and gotchas

- **No native ES modules at runtime.** All `import` is rewritten to `require` by esbuild. Side-effect-only imports run the module body exactly once per Run.
- **No top-level await inside required modules.** Wrap with `(async () => … )()` inside the module if you need it.
- **No `tsconfig.json` honoured.** Module resolution and type checking are independent from any project-level TS config.
- **Per-Run runtime.** Side effects on `globalThis` do not persist between calls to `Run` on the same `Engine`. The require *compile* cache is shared; module *exports* are not.
- **Multi-line ESM imports may stress the entry rewriter.** The current rewriter is line-based and handles the common forms; especially gnarly inputs (comments inside the spec list,

very unusual whitespace) fall back to executing as-is, which may throw.

- **`RegisterConstructor` is d.ts-only today.** At runtime it behaves like `Register`. True `new`-able constructor semantics are on the roadmap.
- **HTTP bindings are real network calls.** `api.http.*` uses `net/http` with a 5s timeout. They are not mockable from JS.

See OUT-OF-SCOPE.md for the active backlog of deferred ideas.

This manual covers sercon v0.4.10. Whenever you add, remove, or change a flag, a binding, or the script API, update this file alongside the help screen (`--help`), the examples walkthrough (`--examples`), and the `CHANGELOG.md`.